

STUDENT LABORATORY MANUAL

Module 1: Introduction to Computer Graphics and Visual Computing

20 Guided Examples • 20 Practice Exercises

Course: CS0045 – Computer Graphics and Visual Computing

Companion: Module 1 Student Guide

Toolchain: VS Code + MSYS2 UCRT64 GCC + FreeGLUT

How to Use This Manual

This manual has three parts:

- Part A — 20 worked Examples (Ex01–Ex20). Each is a complete, compiled, ready-to-run program that demonstrates one concept at a time. Type or copy each one into your SOURCE CODE\M1 folder, compile it (Ctrl+Shift+B), and run it (F5) before moving to the next.
- Part B — 20 Practice Exercises (Q01–Q20), split into Easy (Q01–Q07), Medium (Q08–Q14), and Hard (Q15–Q20). Read the task and hints, then write the program YOURSELF before checking Part C.
- Part C — the full Answer Key for every exercise in Part B, with complete, compiled, working code.

Setup reminder

Use the shared SOURCE CODE\ VS Code workspace described in the Student Guide (Section 4). Compile with Ctrl+Shift+B ("Build OpenGL Program"), then run with F5 or from the terminal. All programs in this manual use the same cross-platform header block shown below, so they compile on Windows (MSYS2 UCRT64 + FreeGLUT), macOS, and Linux.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;
```

Every program in this manual has been compiled and verified with g++ against FreeGLUT.

Part A — Guided Examples (Ex01–Ex20)

Example 01 — The Empty GLUT Window

Concept: The minimal GLUT program skeleton; clearing the framebuffer with no geometry.

Expected output: A window filled entirely with a dark blue background — no shapes yet.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 01 - The Empty GLUT Window
 * -----
 * Concept: OpenGL / GLUT program skeleton.
 * Pipeline stage highlighted: Per-Sample Operations (the framebuffer
 * is cleared to a background color before anything else happens).
 *
 * This is the smallest possible OpenGL/GLUT program. It opens a
 * window and clears it to a solid color. There is no geometry yet -
 * this proves that the pipeline runs even with zero primitives.
 */

void display() {
    glClearColor(0.1f, 0.1f, 0.35f, 1.0f); // dark blue background
    glClear(GL_COLOR_BUFFER_BIT);
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex01 - Empty Window");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Example 02 — Drawing a Single Point

Concept: GL_POINTS primitive mode; glPointSize.

Expected output: One large white dot at the center of the window.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 02 - Drawing a Single Point
 * -----
 * Concept: Vertex Specification + Primitive Assembly (GL_POINTS).
 * A "vertex" is just a coordinate. GL_POINTS tells the Primitive
 * Assembly stage to treat every vertex as its own primitive (a dot).
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1.0f, 1.0f, 1.0f); // white point
    glPointSize(10.0f);          // make the point big enough to see

    glBegin(GL_POINTS);
        glVertex2f(0.0f, 0.0f); // the origin, dead center of the window
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex02 - Single Point");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 03 — Drawing a Single Line

Concept: GL_LINES primitive mode; glLineWidth.

Expected output: A single diagonal yellow line segment.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else

```

```

#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 03 - Drawing a Single Line
 * -----
 * Concept: GL_LINES primitive mode.
 * GL_LINES groups vertices in PAIRS: (v0,v1), (v2,v3), ...
 * Each pair becomes one independent line segment.
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 0.0f); // yellow
    glLineWidth(3.0f);

    glBegin(GL_LINES);
        glVertex2f(-0.7f, -0.5f);
        glVertex2f(0.7f, 0.5f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex03 - Single Line");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 04 — GL_LINE_STRIP

Concept: Connecting vertices in an open chain.

Expected output: A cyan zig-zag line made of connected segments.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

```

```

/*
 * Example 04 - GL_LINE_STRIP
 * -----
 * Concept: GL_LINE_STRIP connects each vertex to the NEXT one in a
 * continuous chain (v0-v1, v1-v2, v2-v3, ...). The shape does NOT
 * close back to the first vertex.
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0f, 1.0f, 1.0f); // cyan
    glLineWidth(2.0f);

    glBegin(GL_LINE_STRIP);
        glVertex2f(-0.8f, -0.4f);
        glVertex2f(-0.4f, 0.4f);
        glVertex2f(0.0f, -0.4f);
        glVertex2f(0.4f, 0.4f);
        glVertex2f(0.8f, -0.4f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex04 - Line Strip (Zig-Zag)");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 05 — GL_LINE_LOOP

Concept: Closing a chain of vertices back to the start.

Expected output: A pink triangle OUTLINE (not filled).

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

```

```

/*
 * Example 05 - GL_LINE_LOOP
 * -----
 * Concept: GL_LINE_LOOP behaves like GL_LINE_STRIP, but it ADDS one
 * extra segment connecting the last vertex back to the first,
 * producing a closed outline (here, a triangle outline).
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 0.4f, 0.7f); // pink
    glLineWidth(2.0f);

    glBegin(GL_LINE_LOOP);
        glVertex2f( 0.0f,  0.6f);
        glVertex2f(-0.6f, -0.4f);
        glVertex2f( 0.6f, -0.4f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex05 - Line Loop (Triangle Outline)");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 06 — A Filled Triangle

Concept: GL_TRIANGLES primitive mode and rasterization.

Expected output: A solid green filled triangle.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 06 - A Filled Triangle
 * -----

```

```

* Concept: GL_TRIANGLES primitive mode + Rasterization.
* Three vertices become one filled triangle. Rasterization is the
* pipeline stage that converts this triangle into fragments (pixels).
*/

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0f, 1.0f, 0.0f); // green

    glBegin(GL_TRIANGLES);
        glVertex2f( 0.0f,  0.6f);
        glVertex2f(-0.6f, -0.4f);
        glVertex2f( 0.6f, -0.4f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex06 - Filled Triangle");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 07 — A Filled Square (GL_POLYGON)

Concept: GL_POLYGON — the exact shape from the lecture slides.

Expected output: A solid white filled square on a black background.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
* Example 07 - A Filled Square (GL_POLYGON)
* -----
* Concept: GL_POLYGON fills the interior bounded by an ordered list
* of vertices. This is the exact shape from the Module 1 slides.
*/

```



```

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_POLYGON);
        glVertex2f(-0.5f, -0.5f);
        glVertex2f(-0.5f, 0.5f);
        glVertex2f( 0.5f, 0.5f);
        glVertex2f( 0.5f, -0.5f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex07 - Filled Square");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 08 — Changing the Background Color

Concept: `glClearColor` sets the color used by the next `glClear`.

Expected output: A peach background with a dark gray square in the center.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 08 - Changing the Background Color
 * -----
 * Concept: glClearColor(r, g, b, a) sets the color used the NEXT
 * time glClear(GL_COLOR_BUFFER_BIT) runs. It does not draw anything
 * by itself - it just "loads the paint" for the clear operation.
 */

void display() {
    glClearColor(1.0f, 0.85f, 0.6f, 1.0f); // warm peach background
    glClear(GL_COLOR_BUFFER_BIT);
}

```

```

    glColor3f(0.2f, 0.2f, 0.2f); // dark gray square
    glBegin(GL_POLYGON);
        glVertex2f(-0.4f, -0.4f);
        glVertex2f(-0.4f, 0.4f);
        glVertex2f(0.4f, 0.4f);
        glVertex2f(0.4f, -0.4f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex08 - Custom Background Color");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 09 — Coloring a Shape (glColor3f)

Concept: The CURRENT color state and how it applies to later vertices.

Expected output: A red square on the left and a blue square on the right.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 09 - Coloring a Shape (glColor3f)
 * -----
 * Concept: glColor3f(r, g, b) sets the CURRENT color. Every vertex
 * drawn afterward uses that color until glColor3f is called again.
 * This is a simplified stand-in for what a fragment shader would do
 * in the modern programmable pipeline.
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.9f, 0.1f, 0.1f); // red square, left side
    glBegin(GL_POLYGON);

```

```

        glVertex2f(-0.8f, -0.3f);
        glVertex2f(-0.8f,  0.3f);
        glVertex2f(-0.2f,  0.3f);
        glVertex2f(-0.2f, -0.3f);
    glEnd();

    glColor3f(0.1f, 0.3f, 0.9f); // blue square, right side
    glBegin(GL_POLYGON);
        glVertex2f( 0.2f, -0.3f);
        glVertex2f( 0.2f,  0.3f);
        glVertex2f( 0.8f,  0.3f);
        glVertex2f( 0.8f, -0.3f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex09 - Two Colored Squares");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 10 — Per-Vertex Color / Smooth Shading

Concept: Different colors per vertex are interpolated across a primitive.

Expected output: A triangle that blends smoothly from red to green to blue.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 10 - Per-Vertex Color / Smooth Shading
 * -----
 * Concept: When each vertex of a primitive gets a DIFFERENT color,
 * OpenGL interpolates the colors across the interior during
 * Rasterization. This mimics how the Vertex Shader stage passes
 * varying data down to the Fragment Shader stage in the modern
 * pipeline.
 */

```

```

*/

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glShadeModel(GL_SMOOTH);
    glBegin(GL_TRIANGLES);
        glColor3f(1.0f, 0.0f, 0.0f); glVertex2f( 0.0f,  0.6f); // red top
        glColor3f(0.0f, 1.0f, 0.0f); glVertex2f(-0.6f, -0.4f); // green left
        glColor3f(0.0f, 0.0f, 1.0f); glVertex2f( 0.6f, -0.4f); // blue right
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex10 - Smooth Shaded Triangle");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 11 — Controlling Window Size and Position

Concept: glutInitWindowSize / glutInitWindowPosition, called before glutCreateWindow.

Expected output: A wide 1024x768 window positioned near the screen's top-left, with a square outline.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 11 - Controlling Window Size and Position
 * -----
 * Concept: glutInitWindowSize / glutInitWindowPosition let you
 * control how big the window is and where it appears on screen
 * BEFORE glutCreateWindow is called.
 */

void display() {
    glClearColor(0.95f, 0.95f, 0.95f, 1.0f);

```

```

    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0f, 0.0f, 0.0f);
    glBegin(GL_LINE_LOOP);
        glVertex2f(-0.9f, -0.9f);
        glVertex2f(-0.9f, 0.9f);
        glVertex2f(0.9f, 0.9f);
        glVertex2f(0.9f, -0.9f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(1024, 768);    // wide window
    glutInitWindowPosition(50, 50);   // near the top-left of the screen
    glutCreateWindow("Ex11 - Custom Size (1024x768) and Position");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 12 — Visualizing the Coordinate System

Concept: Default OpenGL coordinates span -1..1 on X and Y, origin centered.

Expected output: Black X/Y axes with tick marks and a single red point at (0.5, 0.5).

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 12 - Visualizing the OpenGL Coordinate System
 * -----
 * Concept: By default, OpenGL's visible area (before any projection
 * changes) spans from -1.0 to 1.0 on both X and Y, with (0,0) at the
 * CENTER of the window. This program draws the X and Y axes plus tick
 * marks so students can see that layout directly.
 */

void drawAxes() {
    glColor3f(0.0f, 0.0f, 0.0f);

```

```

glLineWidth(2.0f);
glBegin(GL_LINES);
    // X axis
    glVertex2f(-1.0f, 0.0f);
    glVertex2f( 1.0f, 0.0f);
    // Y axis
    glVertex2f(0.0f, -1.0f);
    glVertex2f(0.0f,  1.0f);
glEnd();

// small tick marks every 0.25 units on the X axis
glBegin(GL_LINES);
for (float x = -1.0f; x <= 1.0f; x += 0.25f) {
    glVertex2f(x, -0.02f);
    glVertex2f(x,  0.02f);
}
glEnd();
}

void display() {
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    drawAxes();

    // a point marking a sample coordinate (0.5, 0.5)
    glColor3f(1.0f, 0.0f, 0.0f);
    glPointSize(8.0f);
    glBegin(GL_POINTS);
        glVertex2f(0.5f, 0.5f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex12 - Coordinate System Axes");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 13 — Multiple Independent Shapes

Concept: Several glBegin/glEnd blocks inside one display().

Expected output: A triangle, a square, and a line, each a different color, in one window.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 13 - Multiple Independent Shapes in One Scene
 * -----
 * Concept: display() may contain several glBegin()/glEnd() blocks.
 * Each block is submitted through the pipeline separately, but they
 * all land in the same framebuffer before glFlush() is called.
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // a triangle on the left
    glColor3f(0.9f, 0.6f, 0.0f);
    glBegin(GL_TRIANGLES);
        glVertex2f(-0.9f, -0.3f);
        glVertex2f(-0.5f, 0.5f);
        glVertex2f(-0.1f, -0.3f);
    glEnd();

    // a square in the middle
    glColor3f(0.0f, 0.6f, 0.3f);
    glBegin(GL_POLYGON);
        glVertex2f(-0.2f, -0.3f);
        glVertex2f(-0.2f, 0.3f);
        glVertex2f(0.2f, 0.3f);
        glVertex2f(0.2f, -0.3f);
    glEnd();

    // a line on the right
    glColor3f(0.2f, 0.2f, 0.9f);
    glLineWidth(4.0f);
    glBegin(GL_LINES);
        glVertex2f(0.4f, -0.4f);
        glVertex2f(0.9f, 0.4f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(700, 500);
    glutCreateWindow("Ex13 - Multiple Shapes in One Window");
}

```

```

    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 14 — glPointSize and a Field of Points

Concept: Using nested loops to generate many vertices programmatically.

Expected output: A grid of evenly spaced blue points.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 14 - glPointSize and a Field of Points
 * -----
 * Concept: glPointSize controls how many pixels (fragments) each
 * GL_POINTS vertex covers after Rasterization.
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.1f, 0.5f, 1.0f);
    glPointSize(6.0f);

    glBegin(GL_POINTS);
    for (float x = -0.8f; x <= 0.8f; x += 0.2f) {
        for (float y = -0.8f; y <= 0.8f; y += 0.2f) {
            glVertex2f(x, y);
        }
    }
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex14 - Grid of Points");
    glutDisplayFunc(display);
    glutMainLoop();
}

```



```
    return 0;
}
```

Example 15 — glLineWidth and a Line Grid

Concept: Loops generating GL_LINES for a grid pattern.

Expected output: A gray grid of horizontal and vertical lines.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 15 - glLineWidth and a Line Grid
 * -----
 * Concept: glLineWidth changes how thick GL_LINES primitives are
 * once rasterized. Loops let us generate many vertices programmatically
 * instead of typing each glVertex2f call by hand.
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.5f, 0.5f, 0.5f);
    glLineWidth(1.5f);

    glBegin(GL_LINES);
    for (float x = -0.8f; x <= 0.8f; x += 0.2f) {
        glVertex2f(x, -0.8f);
        glVertex2f(x, 0.8f);
    }
    for (float y = -0.8f; y <= 0.8f; y += 0.2f) {
        glVertex2f(-0.8f, y);
        glVertex2f(0.8f, y);
    }
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex15 - Line Grid");
}
```

```

    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 16 — A Regular Pentagon

Concept: Building a regular polygon with sin/cos around a circle.

Expected output: A solid purple pentagon.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 16 - A Regular Pentagon
 * -----
 * Concept: Any regular polygon can be built by placing vertices
 * around a circle at equal angle steps, then feeding them to
 * GL_POLYGON in order.
 */
#include <cmath>

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.6f, 0.2f, 0.8f);

    const int sides = 5;
    const float radius = 0.6f;

    glBegin(GL_POLYGON);
    for (int i = 0; i < sides; i++) {
        float angle = (float)i / sides * 2.0f * 3.14159265f + 1.5707963f; // start
        pointing up
        float x = radius * cosf(angle);
        float y = radius * sinf(angle);
        glVertex2f(x, y);
    }
    glEnd();

    glFlush();
}

```

```

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex16 - Regular Pentagon");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Example 17 — Approximating a Circle (GL_TRIANGLE_FAN)

Concept: A shared center vertex plus many surrounding vertices approximates a circle.

Expected output: A solid orange circle.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 17 - Approximating a Circle with GL_TRIANGLE_FAN
 * -----
 * Concept: GL_TRIANGLE_FAN treats the first vertex as a shared
 * "center" vertex; every pair of following vertices forms a triangle
 * with that center. Enough thin triangles around a center point look
 * like a filled circle once rasterized.
 */
#include <cmath>

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 0.6f, 0.0f);

    const int segments = 60;
    const float radius = 0.6f;

    glBegin(GL_TRIANGLE_FAN);
        glVertex2f(0.0f, 0.0f); // center of the fan
        for (int i = 0; i <= segments; i++) {
            float angle = (float)i / segments * 2.0f * 3.14159265f;
            glVertex2f(radius * cosf(angle), radius * sinf(angle));
        }
    glEnd();
}

```

```

        glFlush();
    }

    int main(int argc, char** argv) {
        glutInit(&argc, argv);
        glutInitWindowSize(600, 600);
        glutCreateWindow("Ex17 - Circle via Triangle Fan");
        glutDisplayFunc(display);
        glutMainLoop();
        return 0;
    }

```

Example 18 — Responding to Keyboard Input

Concept: glutKeyboardFunc and glutPostRedisplay.

Expected output: A square that changes color when you press r, g, or b; ESC quits.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 18 - Responding to Keyboard Input
 * -----
 * Concept: glutKeyboardFunc registers a callback that GLUT invokes
 * whenever a key is pressed. Changing a global variable and calling
 * glutPostRedisplay() tells GLUT "the scene changed, please call
 * display() again."
 */

float shapeColor[3] = {1.0f, 0.0f, 0.0f}; // starts red

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(shapeColor[0], shapeColor[1], shapeColor[2]);

    glBegin(GL_POLYGON);
        glVertex2f(-0.5f, -0.5f);
        glVertex2f(-0.5f, 0.5f);
        glVertex2f(0.5f, 0.5f);
        glVertex2f(0.5f, -0.5f);
    glEnd();
}

```

```

        glFlush();
    }

    void keyboard(unsigned char key, int x, int y) {
        if (key == 'r') { shapeColor[0] = 1; shapeColor[1] = 0; shapeColor[2] = 0; }
        if (key == 'g') { shapeColor[0] = 0; shapeColor[1] = 1; shapeColor[2] = 0; }
        if (key == 'b') { shapeColor[0] = 0; shapeColor[1] = 0; shapeColor[2] = 1; }
        if (key == 27) { exit(0); } // ESC key quits

        glutPostRedisplay(); // ask GLUT to redraw with the new color
    }

    int main(int argc, char** argv) {
        glutInit(&argc, argv);
        glutInitWindowSize(600, 600);
        glutCreateWindow("Ex18 - Press r/g/b to change color, ESC to quit");
        glutDisplayFunc(display);
        glutKeyboardFunc(keyboard);
        glutMainLoop();
        return 0;
    }

```

Example 19 — Simple Animation (glutIdleFunc)

Concept: Continuous redraw driven by an idle callback.

Expected output: A square that slides left and right, bouncing off the window edges.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 19 - Simple Animation with glutIdleFunc
 * -----
 * Concept: glutIdleFunc registers a callback that GLUT calls
 * continuously whenever there are no other events to process. This
 * is how classic GLUT programs animate: update a variable, then
 * request a redraw.
 */

float squareX = -0.8f;
float speed = 0.01f;

```

```

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.2f, 0.7f, 0.9f);

    glBegin(GL_POLYGON);
        glVertex2f(squareX - 0.1f, -0.1f);
        glVertex2f(squareX - 0.1f, 0.1f);
        glVertex2f(squareX + 0.1f, 0.1f);
        glVertex2f(squareX + 0.1f, -0.1f);
    glEnd();

    glFlush();
}

void idle() {
    squareX += speed;
    if (squareX > 0.8f || squareX < -0.8f) {
        speed = -speed; // bounce off the edges
    }
    glutPostRedisplay();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 300);
    glutCreateWindow("Ex19 - Bouncing Square (glutIdleFunc)");
    glutDisplayFunc(display);
    glutIdleFunc(idle);
    glutMainLoop();
    return 0;
}

```

Example 20 — Capstone: A Small Composed Scene

Concept: Combining background color, multiple primitives, per-vertex color, and a triangle-fan circle.

Expected output: A simple landscape: sky, sun, ground, and a house.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 20 - Capstone: A Small Composed Scene

```

```

* -----
* Concept: Combines everything from this module - background color,
* multiple primitive types, per-vertex color, and a circle built
* from a triangle fan - into one simple "landscape" scene: sky,
* ground, sun, and a house.
*/
#include <cmath>

void drawSun() {
    glColor3f(1.0f, 0.85f, 0.2f);
    const int segments = 40;
    glBegin(GL_TRIANGLE_FAN);
        glVertex2f(0.6f, 0.6f);
        for (int i = 0; i <= segments; i++) {
            float angle = (float)i / segments * 2.0f * 3.14159265f;
            glVertex2f(0.6f + 0.18f * cosf(angle), 0.6f + 0.18f * sinf(angle));
        }
    glEnd();
}

void drawGround() {
    glColor3f(0.2f, 0.6f, 0.2f);
    glBegin(GL_POLYGON);
        glVertex2f(-1.0f, -1.0f);
        glVertex2f(-1.0f, -0.3f);
        glVertex2f( 1.0f, -0.3f);
        glVertex2f( 1.0f, -1.0f);
    glEnd();
}

void drawHouse() {
    // walls
    glColor3f(0.8f, 0.5f, 0.3f);
    glBegin(GL_POLYGON);
        glVertex2f(-0.3f, -0.3f);
        glVertex2f(-0.3f,  0.1f);
        glVertex2f( 0.3f,  0.1f);
        glVertex2f( 0.3f, -0.3f);
    glEnd();

    // roof
    glColor3f(0.6f, 0.1f, 0.1f);
    glBegin(GL_TRIANGLES);
        glVertex2f(-0.35f, 0.1f);
        glVertex2f( 0.0f,  0.4f);
        glVertex2f( 0.35f, 0.1f);
    glEnd();
}

void display() {
    glClearColor(0.6f, 0.8f, 1.0f, 1.0f); // sky blue
    glClear(GL_COLOR_BUFFER_BIT);
}

```

```
    drawSun();
    drawGround();
    drawHouse();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(700, 700);
    glutCreateWindow("Ex20 - Mini Scene");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```


Part B — Practice Exercises (Q01–Q20)

Attempt each exercise on your own first. Compile and run your own solution, compare its output to the task description, and only THEN check Part C. Difficulty is marked Easy, Medium, or Hard.

Easy Exercises

Exercise Q01 — Red Center Point [Easy]

Objective: Practice specifying a single vertex and controlling point size.

Task: Write a program that draws one large RED point exactly at the center of the window (point size 15).

Hints:

- Use `GL_POINTS` inside `glBegin/glEnd`.
- Remember `glColor3f` must be set BEFORE the `glBegin` call to affect that primitive.
- `glPointSize(15.0f)` makes the point large enough to see clearly.

Exercise Q02 — Horizontal Green Line [Easy]

Objective: Practice `GL_LINES` and `glLineWidth`.

Task: Draw a single thick GREEN horizontal line that spans most of the width of the window.

Hints:

- `GL_LINES` needs exactly two vertices for one segment.
- Keep both vertices at the same Y value for a horizontal line.
- Try `glLineWidth(4.0f)`.

Exercise Q03 — Triangle Outline [Easy]

Objective: Practice `GL_LINE_LOOP` and distinguish outlines from filled shapes.

Task: Draw the OUTLINE only (not filled) of a triangle, in blue, using `GL_LINE_LOOP`.

Hints:

- `GL_LINE_LOOP` automatically closes the shape back to the first vertex.
- Compare this visually to Example 6 (filled triangle) once done.

Exercise Q04 — Orange Rectangle [Easy]

Objective: Practice GL_POLYGON with a non-square shape.

Task: Draw a FILLED orange rectangle that is clearly wider than it is tall.

Hints:

- Use four vertices in order (e.g., going counter-clockwise) inside GL_POLYGON.
- Make the X span larger than the Y span.

Exercise Q05 — Square Outline on Gray Background [Easy]

Objective: Practice glClearColor together with GL_LINE_LOOP.

Task: Set the background to light gray, then draw a BLACK square OUTLINE (not filled) in the center of the window.

Hints:

- glClearColor only takes effect the next time glClear(GL_COLOR_BUFFER_BIT) runs.
- Use GL_LINE_LOOP with four vertices for the outline.

Exercise Q06 — Hexagon Outline [Easy]

Objective: Practice manually typing coordinates for a polygon with more than four sides.

Task: Draw the OUTLINE of a six-sided hexagon by typing six glVertex2f calls by hand (no trigonometry) inside GL_LINE_LOOP.

Hints:

- Sketch the hexagon on paper first and estimate coordinates between -1 and 1.
- Go around the shape in one consistent direction (clockwise or counter-clockwise).

Exercise Q07 — Two Triangles [Easy]

Objective: Practice using multiple glBegin/glEnd blocks with different colors in one display().

Task: Draw two separate filled triangles side by side: one RED on the left, one BLUE on the right.

Hints:

- Each triangle needs its own glBegin(GL_TRIANGLES) ... glEnd() block.
- Change the current color with glColor3f between the two blocks.

Medium Exercises

Exercise Q08 — Smooth Shaded Diamond [Medium]

Objective: Practice per-vertex color and smooth shading on a 4-sided polygon.

Task: Draw a diamond (a square rotated 45 degrees) using `GL_POLYGON` where each of the 4 vertices has a DIFFERENT color, so the interior blends smoothly.

Hints:

- Diamond vertices: top, right, bottom, left (each at distance ~ 0.7 from center).
- Call `glColor3f` immediately before each `glVertex2f`, inside the same `glBegin/glEnd` block.
- `glShadeModel(GL_SMOOTH)` enables the blending (it is the default in most implementations, but set it explicitly).

Exercise Q09 — 4x4 Checkerboard [Medium]

Objective: Practice nested loops to generate a grid of filled polygons.

Task: Draw a 4x4 checkerboard pattern using nested for loops, alternating between two colors based on $(\text{row} + \text{col})$ being even or odd.

Hints:

- Use two nested loops: outer for rows, inner for columns.
- Compute each cell's four corner coordinates from its row/column index.
- Use the modulo operator $(\text{row} + \text{col}) \% 2$ to decide the color.

Exercise Q10 — Five-Pointed Star Outline [Medium]

Objective: Practice trigonometry (sin/cos) to generate a non-trivial shape.

Task: Draw the OUTLINE of a five-pointed star using a loop that alternates between an outer radius and a smaller inner radius as it walks around a circle.

Hints:

- A star with 5 points needs 10 vertices total (5 outer + 5 inner), alternating.
- Use `#include <cmath>` for `sinf/cosf`.
- Start the first vertex pointing straight up by offsetting the angle by 90 degrees ($\text{PI}/2$ radians).

Exercise Q11 — Concentric Circles [Medium]

Objective: Practice reusing a helper function to draw the same shape at different sizes.

Task: Draw three circle OUTLINES (different radii, different colors) that share the same center, using `GL_LINE_LOOP` and a loop with `sin/cos`.

Hints:

- Write a helper function `drawCircleOutline(radius, r, g, b)` and call it three times.
- Use at least 40-60 segments so each circle looks smooth.

Exercise Q12 — Arrow [Medium]

Objective: Practice combining two different primitive types (a line and a triangle) into one composite shape.

Task: Draw an arrow pointing to the right: a line for the shaft and a filled triangle for the arrowhead.

Hints:

- Draw the shaft first with `GL_LINES`.
- The arrowhead triangle's base should line up with the end of the shaft.

Exercise Q13 — Letter "F" from Rectangles [Medium]

Objective: Practice building compound shapes ('pixel art' style) from several filled rectangles.

Task: Draw the capital letter "F" using exactly three filled rectangles (a vertical stem, a top bar, and a middle bar).

Hints:

- Write a helper function `drawRect(x0, y0, x1, y1)` that draws one filled rectangle with `GL_POLYGON`, and call it three times.
- Sketch the letter on graph paper first to plan your rectangle coordinates.

Exercise Q14 — Simple Landscape [Medium]

Objective: Practice combining `glutInitWindowSize/Position`, `glClearColor`, `GL_POLYGON`, and `GL_TRIANGLE_FAN` in one scene.

Task: Create an 800x400 window positioned at (100,100) that shows a light-blue sky, a green ground rectangle across the bottom, and a sun (a small circle drawn with `GL_TRIANGLE_FAN`).

Hints:

- Set the window size and position BEFORE `glutCreateWindow`.
- The sky color comes from `glClearColor`; the ground is a separate `GL_POLYGON` rectangle drawn on top of it.

Hard Exercises

Exercise Q15 — Move a Square with Arrow Keys [Hard]

Objective: Practice `glutSpecialFunc` and updating state in response to input.

Task: Draw a small square that the user can move around the window using the ARROW KEYS, without letting it move outside the window bounds.

Hints:

- `glutSpecialFunc` registers a callback for special keys like `GLUT_KEY_LEFT/RIGHT/UP/DOWN`.
- Store the square's center position in global variables and rebuild its four corners from that position inside `display()`.
- After changing position, clamp it so the square can't leave the `-1..1` range, then call `glutPostRedisplay()`.

Exercise Q16 — Bouncing Ball [Hard]

Objective: Practice `glutIdleFunc`-driven animation with collision against all four window edges.

Task: Animate a filled circle (ball) that moves diagonally and bounces off all four edges of the window, reversing direction on the axis it collided with.

Hints:

- Track `ballX`, `ballY`, `dx`, `dy` as global variables.
- In `idle()`, add `dx/dy` to the position, then check if the ball's edge has crossed `-1` or `1` on either axis and flip the corresponding direction.
- Redraw the circle using `GL_TRIANGLE_FAN` centered on the current ball position.

Exercise Q17 — Traffic Light Simulator [Hard]

Objective: Practice using keyboard input to drive a small state machine and reflect that state visually.

Task: Draw a traffic light housing with three stacked circles (red, yellow, green). Pressing the 'n' key advances to the next active light (red -> yellow -> green -> red ...), with the active light drawn bright and the others dimmed.

Hints:

- Keep an integer state variable (0, 1, or 2) tracking which light is active.
- In `display()`, choose each circle's brightness based on whether its index matches the active state.
- Increment the state modulo 3 inside your keyboard callback, then call `glutPostRedisplay()`.

Exercise Q18 — Rotating Clock Hand [Hard]

Objective: Practice combining a static shape (clock face) with an animated one (hand) driven by `glutIdleFunc`.

Task: Draw a clock face: a circle outline with 12 tick marks, plus one hand (a line from the center) that slowly rotates using `glutIdleFunc`.

Hints:

- Draw the static face first (circle + 12 tick marks placed with `sin/cos` at 30-degree increments).
- Track a single angle variable for the hand; decrease or increase it slightly every `idle()` call.
- The hand's endpoint is $(\text{radius} \cdot \cos(\text{angle}), \text{radius} \cdot \sin(\text{angle}))$.

Exercise Q19 — Keyboard Color Picker [Hard]

Objective: Practice arrays, keyboard input, and console output alongside OpenGL rendering.

Task: Let the user press number keys 1 through 6 to change the window's background color between six preset colors. Print the name of the newly selected color to the console each time. Pressing ESC should quit the program.

Hints:

- Store the six colors in a 2D array (or six separate arrays) and their names in a matching array of strings.
- In your keyboard callback, map the pressed key ('1'-'6') to an array index using key - '1'.
- Use `cout` from `<iostream>` to print the color name; call `exit(0)` when ESC (ASCII 27) is pressed.

Exercise Q20 — Procedural Striped Flag with Star (Capstone) [Hard]

Objective: Practice procedural generation: a configurable loop-driven design plus a composite star shape, tying together most of the module's concepts.

Task: Draw a flag made of a CONFIGURABLE number of horizontal stripes (use a named constant such as `STRIPE_COUNT`, generated with a loop — do not hardcode each stripe), alternating between two colors, plus a filled five-pointed star in the upper-left corner.

Hints:

- Compute each stripe's height as $(\text{total height}) / \text{STRIPE_COUNT}$, and its Y range from a loop index.
- Reuse your star-drawing logic from Exercise 10, this time as a filled `GL_POLYGON` instead of an outline.
- Try changing `STRIPE_COUNT` to a different value and recompiling to confirm your loop truly is generic.